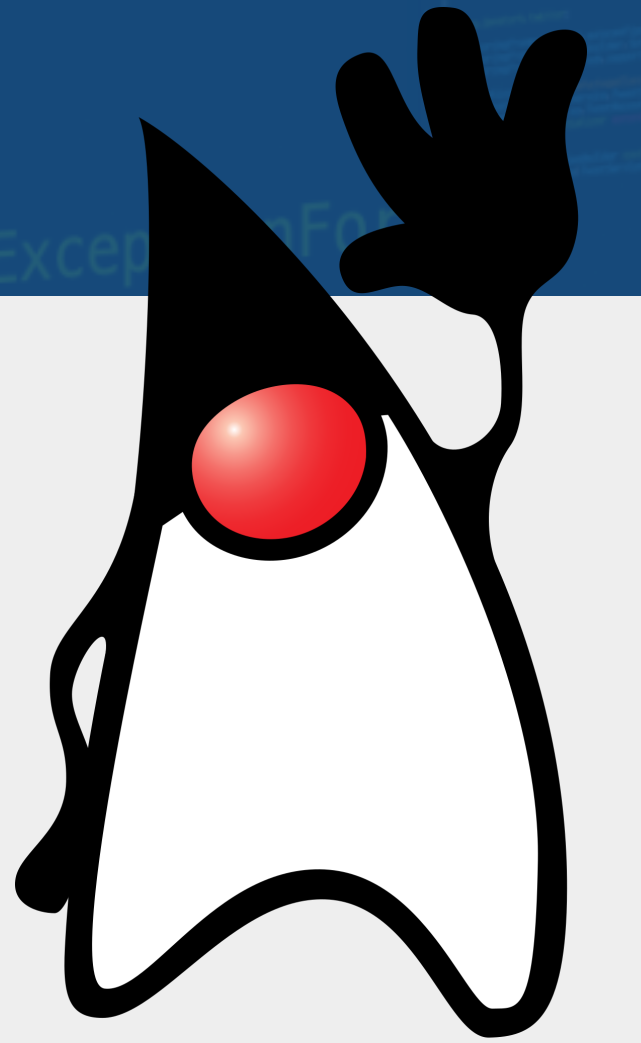


Java Programlama Dili

14. Bölüm

Nesnenin Durumu

Akın Kaldıroğlu



Amaç



- Bu bölümde nesnelerin durumu (state) ele alınacaktır
- Bir sonraki bölümde nesnelerin davranışı işlenecektir



- **Nesnenin Durumu**
 - Nesne Değişkenlerinin İlk Değeri
 - Bileşik Nesneler
 - Stack, Heap ve Değişkenler
- **Ödevler**
- **Bundan Sonra?**

Nesnenin Durumu



Değişkenlerin Roller (Tekrar)



- Java'da değişkenler, ilkel ya da referans, programdaki fonksiyonları ya da rolleri açısından üçe ayrılır:
- **Nesne değişkenleri (instance ya da object variables):** Nesnenin durumunu (state) oluşturan değişkenlerdir
- **Sınıf değişkenleri (class variables):** Nesnelerin ortak durumunu ifade eden değişkenlerdir
- **Yerel değişkenler (local variables):** Geçici değişkenlerdir

Üye Değişkenler



- Nesne ve sınıf değişkenlerine, üye değişkenler denir
- Değişkenlerin tanıtılması ve tanımlanması yanında kapsam, değişmezlik vb. özellikleri daha önce geniş bir şekilde ele alınmıştı
- Üye değişkenlerin kapsamı tüm sınıfken yerel değişkenlerin kapsamı içinde bulundukları blok ile sınırlıdır
- Burada üye değişkenlerden nesne değişkenlerini ele alacağız
- İleriki bir bölümde de sınıf değişkenlerini inceleyeceğiz

Nesne Değişkenleri - I



- **Nesne değişkenleri** (**instance** ya da **object variables**), fonksiyonel olarak, nesnenin özelliklerini ifade eder
 - Nesne değişkenlerine **alan** (**field**) da denir
 - Nesne değişkenlerine bazı durumlarda **özellik** (**property**) de denir
- Nesne değişkenleri, yapısal olarak referans ya da ilkel tipte olabilir
- Eğer nesne değişkeni bir referans tip ise bu durumda o da başka bir sınıftan oluşturulmuş bir nesneyi gösteriyor demektir

Nesne Değişkenleri - II



- Nesne değişkenleri, sınıfın içinde ama metot ya da başlatma bloğu (initializer block) gibi herhangi bir alt blok dışında, herhangi bir yerde tanımlanmalıdır
- Genelde sınıfın en başında tanımlırlar
- Nesne değişkenlerinin kapsamı (scope), tüm sınıftır
- Fakat nesne değişkenlerinde de ilk değer atarken, ileri referans (forward reference) yapılamaz, Değişkenlerin Özellikleri bölümünde ele alınmıştı

Nesne Değişkenleri - III



- `Car` sınıfında 5 tane nesne değişkeni, 4 tane metot vardır
- Hiç sınıf değişkeni yoktur
- Nesne değişkenleri, sınıf bloğunda ve sınıfın başında tanıtılmıştır

```
class Car{  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
  
    double go(int newDistance) {...}  
  
    void accelerate(int newSpeed) {...}  
  
    void stop() {...}  
  
    String info() {...}  
}
```

Nesne Değişkenleri - IV



- Nesne değişkenleri, ilk değerlerini tanıtılırken alabildikleri gibi, daha sonra bir metot içinde, genelde de kurucu metotta alabilirler
- Tanıtılacak nesne değişkeni sayısında bir kısıtlama yoktur
- Ama sağlıklı bir tasarım için onlarca (yüzlerce!) değişkene sahip sınıflardan kaçınılmalıdır
- Sınıf tasarlarken dolayısıyla nesne değişkenlerini belirlerken sayıdan ziyade soyutlanan şeye odaklanılmalıdır



- Belleğiniz elverdiği ölçüde istediğiniz kadar nesne oluşturabilirsiniz
- Önemli olan her nesnenin, programladığınız iş alanındaki yapıları temsil eden, onları soyutlayan **iş nesnelere** (**business object** ya da **entity**) karşılık gelmesidir:
- `Customer`, `Account`, `ValueAddedTax`, `EarlyDeliverable`, `PackageReceived`, `DownpaymentForPolicyPremium`, vb.

Car Sınıfı



- Örneğin yanda sadece özelliklerden oluşan ve hiçbir davranışa sahip olmayan **Car** sınıfı tanıtılmıştır
- **Car**'ın özellikleri, ilkel ve karmaşık tiplerden değişkenlerle betimlenmiştir
- **Car**'ın nesne değişkenleri sadece tanıtılmış, hiçbirine ilk değer atanmamıştır

```
// Attribute only Car
class Car{
    String make;
    String model;
    String year;
    int speed;
    int distance;
}
```

Özelliklere Erişim - I



- Nesnenin özelliklerine nesnenin referansı yoluyla erişilir
- Erişim `.` notasyonuyla olur:

`reference.attribute`

- Bu şekilde erişilen özelliğin değerine ulaşılabileceği gibi tipine uygun atamalar da yapılabilir:

```
Car carObject = new Car();  
carObject.speed = 60;  
System.out.println(carObject.speed);
```

Özelliklere Erişim - II



```
class Car{  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
}
```

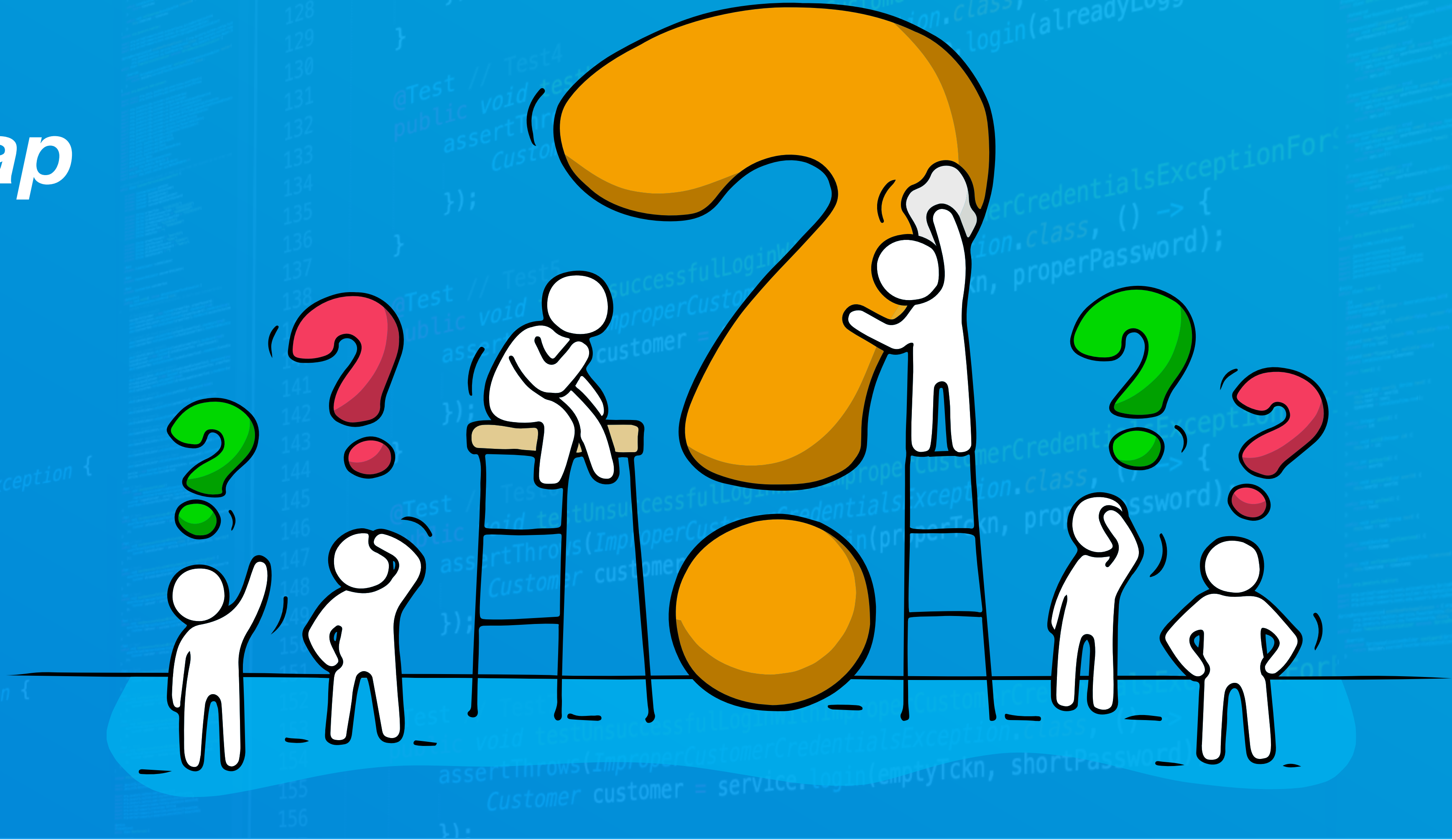
```
class Test {  
  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.make = "Mercedes";  
        myCar.model = "E200";  
        myCar.year = "2019";  
        myCar.speed = 80;  
        myCar.distance = 30_000;  
        System.out.println("My Car: " + myCar.year + " " +  
                             myCar.make + " " + myCar.model);  
  
        Car yourCar = new Car();  
        yourCar.make = "Toyota";  
        yourCar.model = "Camry";  
        yourCar.year = "2011";  
        yourCar.speed = 0;  
        yourCar.distance = 160_000;  
        System.out.println("Your Car: " + yourCar.year + " " +  
                             + yourCar.make + " " + yourCar.model);  
    }  
}
```

Car & Test



- `car.attribute` paketinde `Car & Test`
- Bu örnekteki `Car` sınıfında hiç metot yoktur; sadece nesne özellikleri yani alanlar vardır

Soru ve Cevap Zamanı!





Nesne Değişkenlerinin İlk Değeri

Nesne Değişkenlerinin İlk Değeri - I



- Az önceki örnekte, oluşturulan `Car` nesnesinin alanlarına (field) yani nesne değişkenlerine hiçbir ilk değer verilmediği halde hata alınmadığını hatta varsayılan bir değere sahip olduğunu fark ettiniz mi?
- Ama biz daha önce, Java'da bir değişkene ilk değer atamadan, yani onu tanımlamadan, kullanamayacağımızı söylemiştik

```
Car myCar = new Car();  
  
System.out.println("My Car: " + myCar.year + " " + myCar.make + " " +  
    myCar.model + " " + myCar.speed + " " + myCar.distance);
```

```
My Car: null null null 0 0 // Print out
```

Nesne Değişkenlerinin İlk Değeri - II



- Değişkenler, bir ilk değer atanmadan kullanılamaz
- Java'da nesne değişkenlerine bir ilk değer atamasak bile, nesne oluşturulurken alanlarına **varsayılan bir ilk değer (default initial value)** verilir
- Varsayılan ilk değer, daima değişkenin tipiyle uyumlu ve en az bilgi sağlayan değerdir

Nesne Değişkenlerinin İlk Değeri - II



- Nesne değişkenlerine otomatik olarak atanan ve onlara tipleriyle uyumlu ve en az bilgi sağlayan varsayılan ilk değerler şunlardır:

```
boolean    false
byte       0
char       görülemeyen karakter (int olarak 0)
short      0
int        0
long       0L    (64 bit)
float      0.0F  (32 bit)
double     0.0D  (64 bit)

reference  null
```

InitialValues



- `init.InitialValues`



- **Soru:** Sizce, Java, yerel değişkenlere bir ilk değer vermezken neden nesne değişkenlerine bir ilk değer veriyor?

Nesnenin Durumu - I



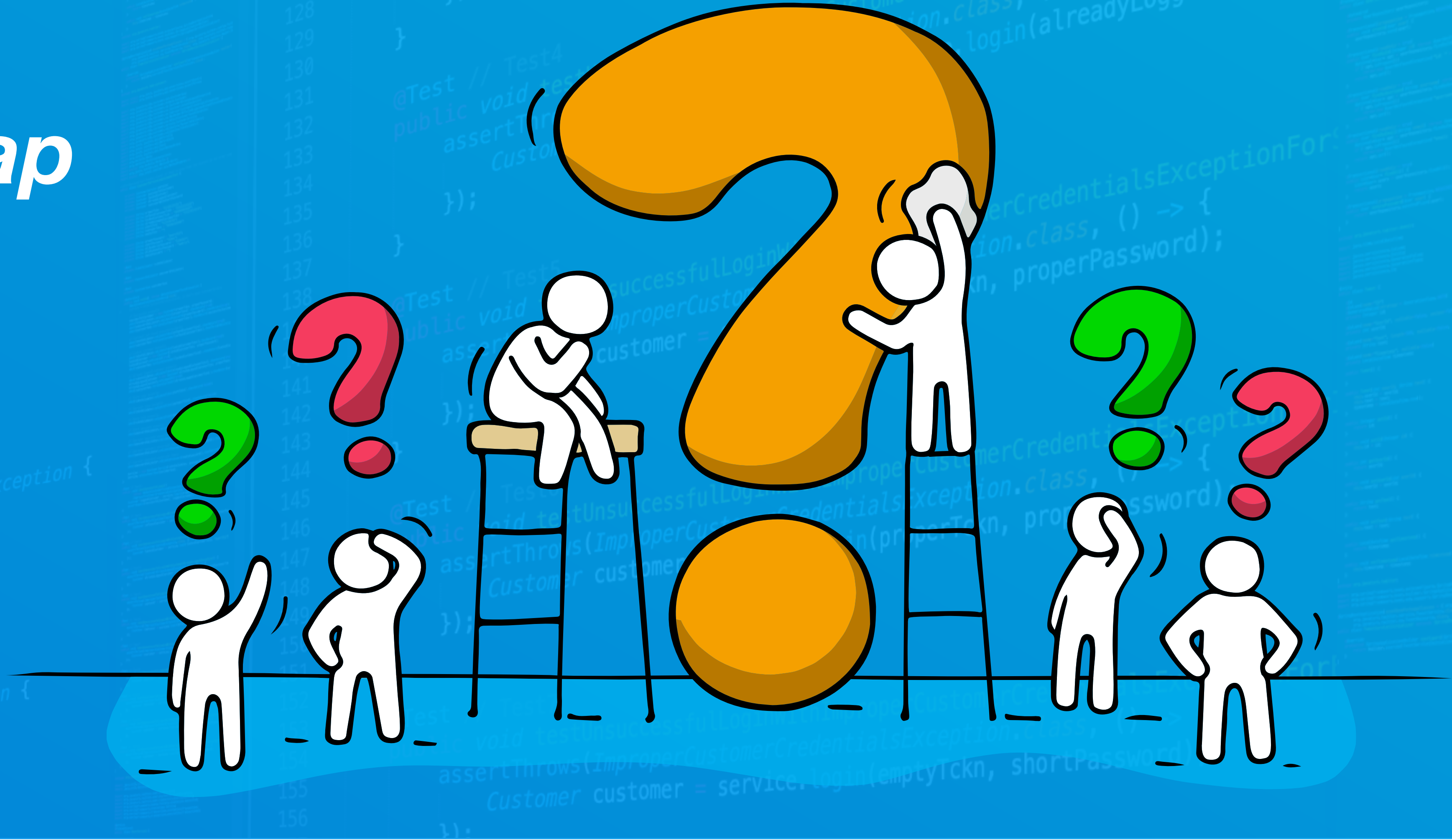
- Java'nın nesne değişkenlerine varsayılan bir ilk değer vermesinin en temel sebebi, nesnenin daima anlamlı bir durumda olmasını sağlama gayretidir
- Nesnelerin durumlarının anlamlı ve tutarlı olması programcının en temel sorumluluklarından
- Java'nın çalışma zamanı yapısı, varsayılan ilk değerlerle buna destek olur

Nesnenin Durumu - II



- Nesnenin durumu, varsayılan ilk değerlere rağmen çoğunlukla anlamsız olacaktır
- Örneğin referansların varsayılan `null` ilk değerleri ya da cinsiyeti gösteren `char` tipinin `0` olan ilk değeri problemlidir
- Nesne durumunun, oluşturulurken anlamlı halde olmasını sağlamanın yolu, akıllı kurucu (smart constructor) çağrısı yapmaktır
- Bunu ileride ele alacağız

Soru ve Cevap Zamanı!





Bileşik Nesneler

Bileşik Nesne - I



- Değişik nesnelerin bir araya getirilmesiyle oluşan nesneye, **bileşik nesne (composite/aggregate object)** denir
- Bu durumda bir sınıf, aynı ya da farklı sınıfların tipinden nesne değişkenlerine sahip olur
- Ya da bir nesne, diğer nesnelerin referanslarına sahip olur

Bileşik Nesne - II



- Daha önce tanımladığımız `Car` sınıfı, `String` tipinden değişkenlere sahipti ve bu yüzden bir bileşik nesneydi
- Fakat bileşik nesne ile biz daha çok, kendi oluşturduğumuz tiplerden nesnelere sahip bileşik nesneleri kastederiz

Car ve Person - I

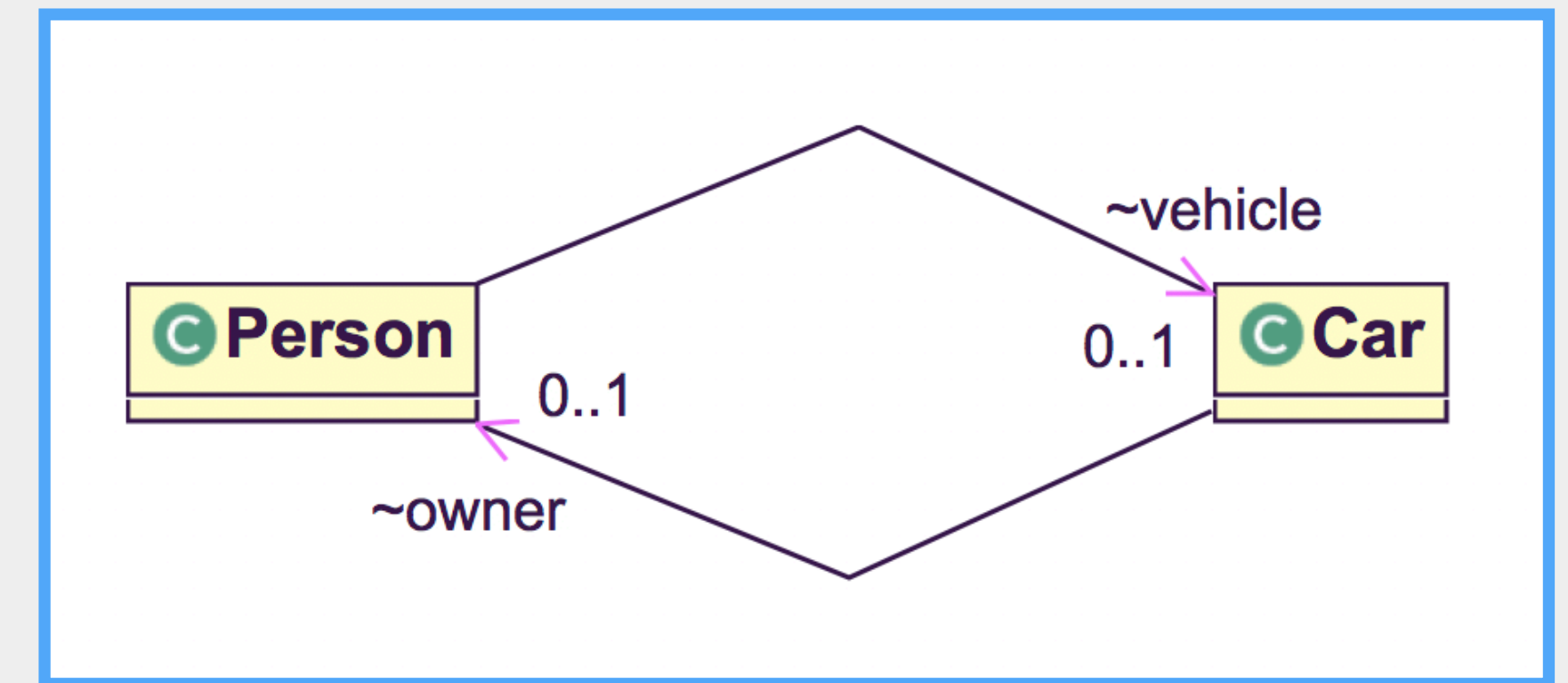


- Örneğin her `Car` nesnesinin bir sahibinin olduğunu düşünün
- Sahipler de `Person` sınıfı ile temsil edilsin
- Bu durumda her `Car` nesnesinin bir `Person` tipinde sahibi ve her `Person` nesnesinin de bir `Car` tipinde aracı varsa, arada **1-1 (bire bir) ilişki (one-to-one association)** var demektir

Car ve Person - II



- Bu durumda **Car** sınıfında **Person** tipinde, örneğin **owner** isimli bir referans
- **Person** sınıfında da **Car** tipinde, örneğin **vehicle** isimli bir referans var olacaktır
- **Person** ile **Car** arasında, **bire-bir ve iki yönlü ilişki (one-to-one, bi-directional association)**, UML'in sınıf diyagramlarında şöyle gösterilir:



```
class Car{
    String make;
    String model;
    String year;
    int speed;
    int distance;
    Person owner;
}
```

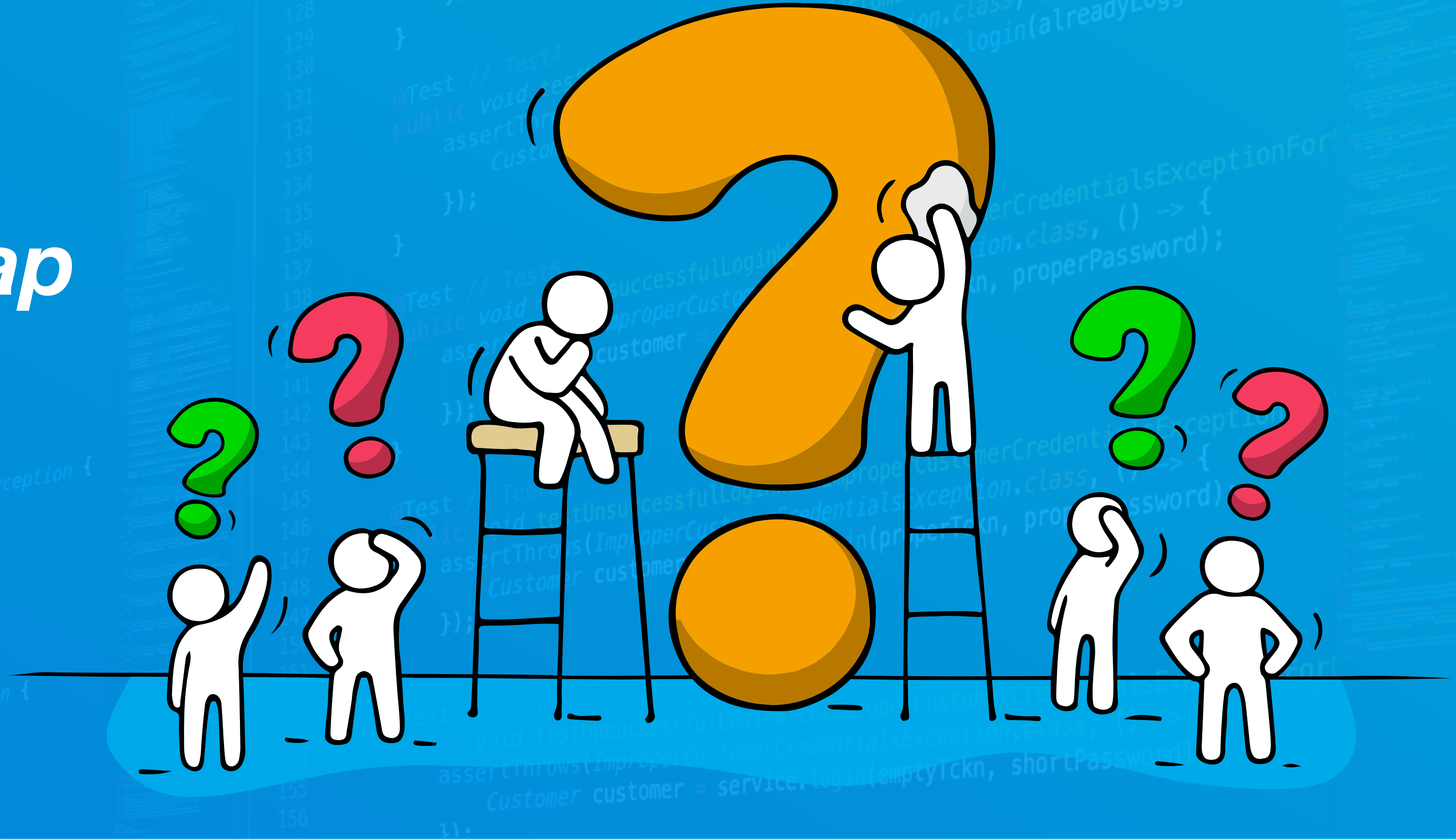
```
class Person{
    String tckn;
    String firstName;
    String lastName;
    Car vehicle;
}
```

Car ve Person



- `car.composite` paketinde `Car` ve `Person`
- Bu örnekteki `Test` sınıfında `Car` ve `Person` nesneleri arasındaki ilişkiye dikkat edin
- `Person` ve `Car` nesnelerindeki referansların heapteyken `Test`'in `main` metodunda oluşturulan referansların stackte, `main`'in bellek alanında olduklarını da gözlemleyin

Soru ve Cevap Zamanı!





Stack, Heap ve Değişkenler

Stack, Heap ve Değişkenler - I



- Daha önce bahsedilmişti, yerel değişkenler (local variables) stackte, nesne değişkenleri ise heapte yaşar
- Çünkü nesneler daima heapte oluşturulur ve orada yaşarlar, nesnenin değişkenleri de tabii olarak heapte, nesnenin bellek alanında bulunur
- Bu durum, değişkenin tipinden bağımsızdır
- Değişken ister referans ister ilkel tipten olsun, bu geçerlidir

Stack, Heap ve Değişkenler - II



- Farklı olan, değişkenin değerinin nerede olduğudur:
 - Eğer değişken ilkel tipten ise değerine doğrudan sahip olur
 - Yerel değişken ise stackte, nesne değişkeni ise heapte, değeriyle birlikte tutulur
 - Eğer değişken referans ise ve `null` değil ise, heapte bulunan bir başka nesneyi gösterir
 - Yerel değişken ise stackte, nesne değişkeni ise heapte bulunur ama her iki halde de heapteki bir nesneyi gösterir

Heap'te Car ve Person - I



- Şimdi bunu **Car** ve **Person** örneğinde görelim:
- **Car** ve **Person** nesnelerinin parçaları olan, **Car** nesnesindeki 6 tane ve **Person** nesnesindeki 4 tane değişken heapte bulunacaktır
- Çünkü **Car**'ın ve **Person**'ın nesneleri tabii olarak heaptedir

```
class Car{  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
    Person owner;  
}
```

```
class Person{  
    String tckn;  
    String firstName;  
    String lastName;  
    Car vehicle;  
}
```

Heap'te Car ve Person - II



- **Test**'in **main**'inde oluşturulan **Person** tipinden **zeynep** ve **Car** tipinden **mercedes** referansları **stack**te, **main** için açılan pencerede (stack frame) bulunacaktır
- Fakat bu referansların gösterdiği **Person** ve **Car** nesneleri, sahip oldukları nesne değişkenleriyle birlikte **heaptedir**

```
...void main(String[] args){  
    Person zeynep = new Person();  
    zeynep.tckn = "1";  
    zeynep.firstName = "Zeynep";  
    zeynep.lastName = "Kaya";  
  
    Car mercedes = new Car();  
    mercedes.make = "Mercedes";  
    mercedes.model = "E200";  
    mercedes.year = "2011";  
    mercedes.speed = 80;  
    mercedes.distance = 30_000;  
    ...  
}
```

Heap'te Car ve Person - III



- Heaptteki **Person** ve **Car** nesnelerinin değişkenlerinin ikisi dışında hepsinin atanmış bir değeri vardır:
- **Person** nesnesinin **Car** tipindeki **vehicle** referansı ile **Car** nesnesinin **Person** tipindeki **owner** referansı, henüz bir atama yapılmadığından **null**'dir
- Ama diğer değişkenlerden referans olanlar yine heaptteki başka nesneleri gösterirken, ilkel olanlar heaptteki değerlerine sahiptir

```
class Person{  
    String tckn;  
    String  
    firstName;  
    String lastName;  
    Car vehicle;  
}
```

```
class Car{  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
    Person owner;  
}
```

```
...void main(String[] args){  
    Person zeynep = new Person();  
    zeynep.tckn = "1";  
    zeynep.firstName = "Zeynep";  
    zeynep.lastName = "Kaya";  
  
    Car mercedes = new Car();  
    mercedes.make = "Mercedes";  
    mercedes.model = "E200";  
    mercedes.year = "2011";  
    mercedes.speed = 80;  
    mercedes.distance = 30_000;  
    ...  
}
```

Heap'te Car ve Person - IV



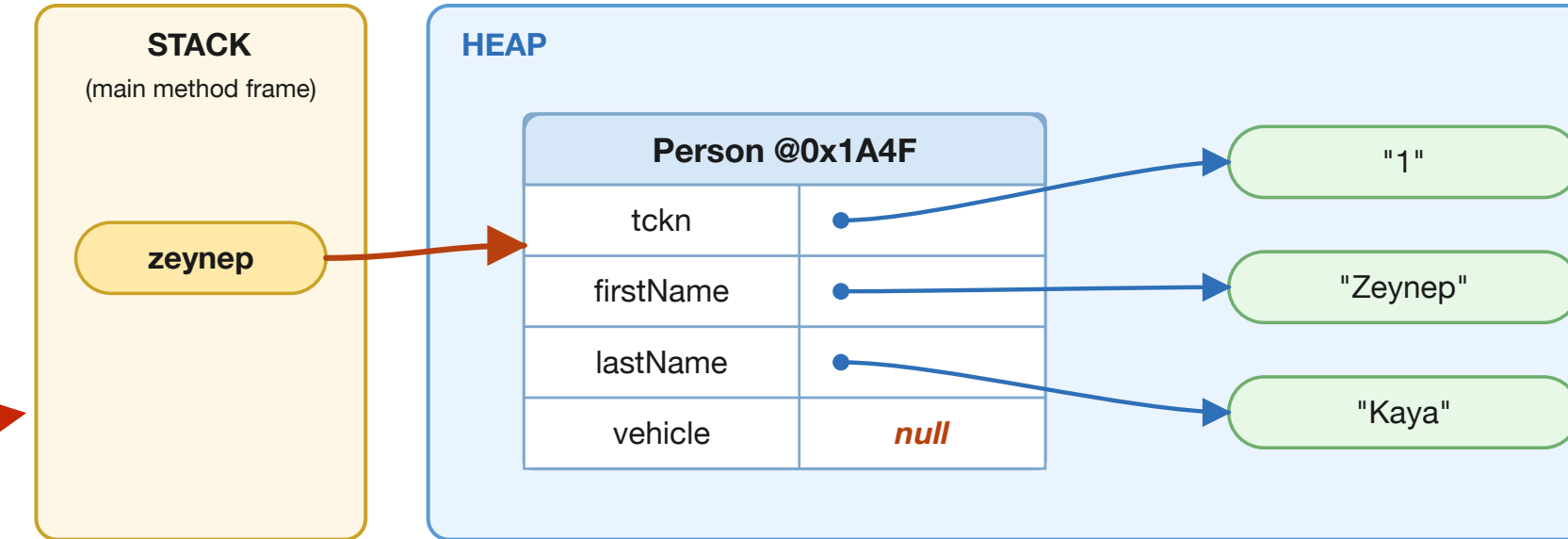
```
... void main(String[] args){  
    Person zeynep = new Person();  
    zeynep.tckn = "1";  
    zeynep.firstName = "Zeynep";  
    zeynep.lastName = "Kaya";
```

// 1

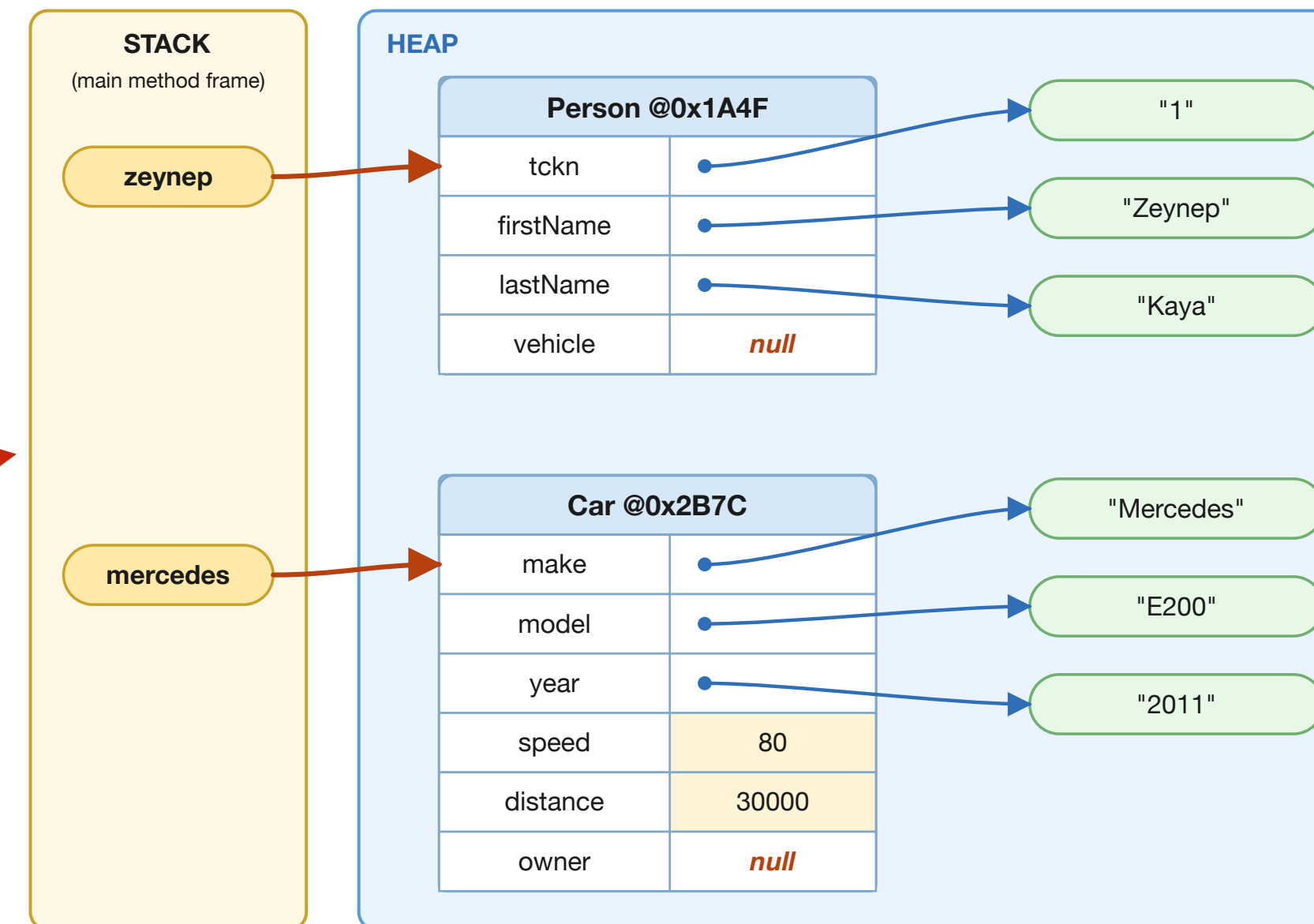
```
    Car mercedes = new Car();  
    mercedes.make = "Mercedes";  
    mercedes.model = "E200";  
    mercedes.year = "2011";  
    mercedes.speed = 80;  
    mercedes.distance = 30_000;
```

// 2

// 1 — yalnızca Person nesnesi var; vehicle henüz null



// 2 — Car nesnesi de yaratıldı; owner ve vehicle hâlâ null

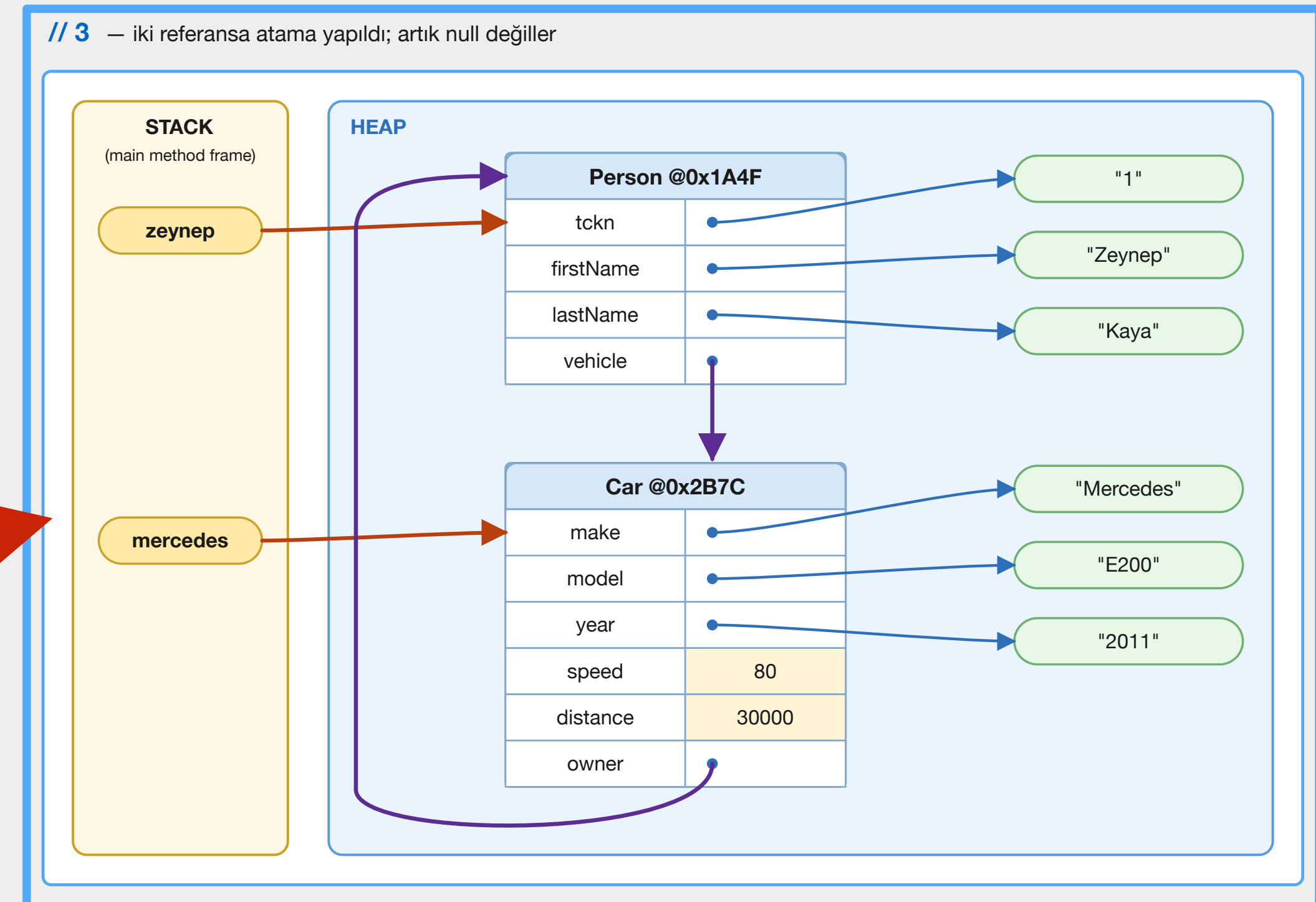


Heap'te Car ve Person - V



- Peki bu iki referansa atama yapıldığında durum ne olur?
- Referanslar `null` olmaktan çıkar ve heapteki nesneleri gösterir
- Aynı, `tckn` ya da `make` referansının, heapteki `String` nesnelerini göstermesi gibi

```
... void main(String[] args){  
    Person zeynep = new Person();  
    ...  
    Car mercedes = new Car();  
    ...  
    mercedes.owner = zeynep;  
    zeynep.vehicle = mercedes;  
    // 3  
    ...  
}
```





Uygulama

Uygulama - I

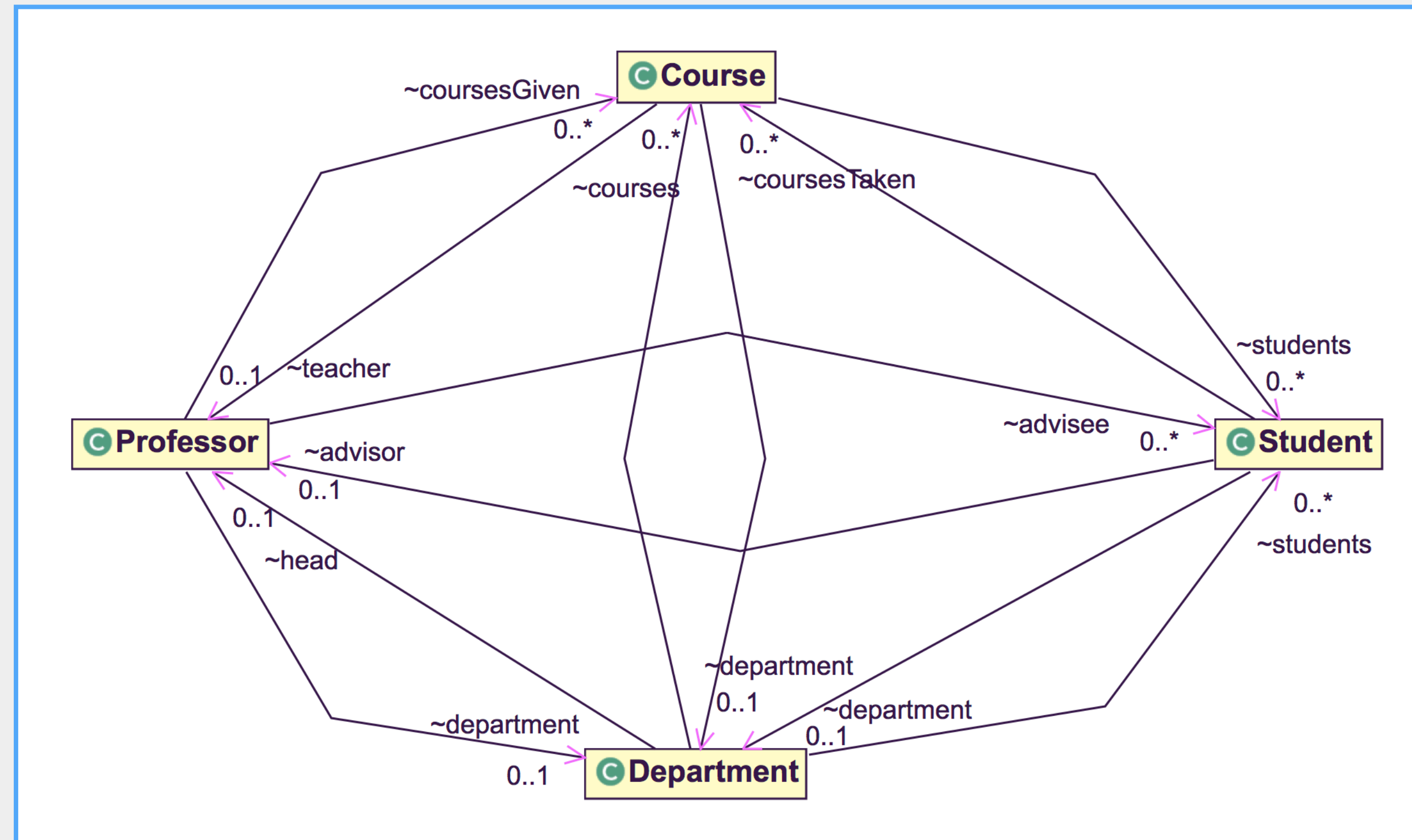


- `university`
- Burada bir üniversite modeli vardır. Nesneleri:
 - `Student`
 - `Course`
 - `Department`
 - `Professor`

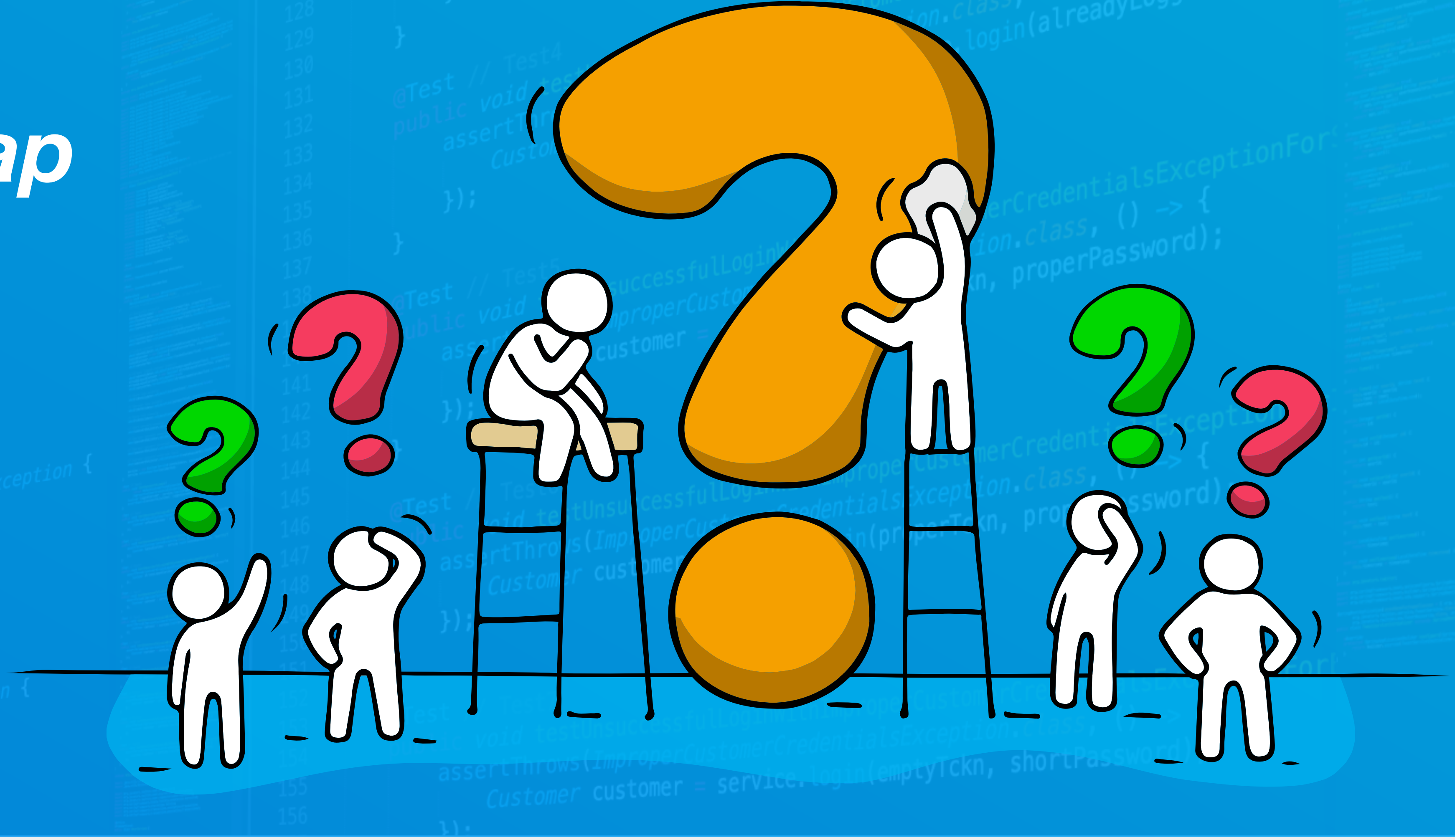
Uygulama - II



- Nesneler arasındaki ilişkiler aşağıdaki UML sınıf diyagramında (class diagram) belirtilmiştir



Soru ve Cevap Zamanı!



Ödevler





1. Üniversite örneğindeki **Test** sınıfının **main** metoduna aşağıdaki eklemeleri yapın:
 - Bir yeni bölüm, örneğin Philosophy
 - Yeni iki profesör, örneğin Ali Veli ve Ayşe Fatma
 - Yeni iki ders, örneğin Ethics ve Ontology
 - Yeni üç öğrenci, Zeynep Gül, Metin Sağlam ve Murat Atılgan
 - Ve bu yeni nesneler arasında ilişkiler kurun



2. İnsanların besledikleri köpekleri nasıl modellersiniz? Sadece alanlarla modellerseniz, hangi alanların olmasını beklersiniz? Alanların isimleri ve tipleri ne olur?
 - a. Uygun sınıfı seçtiğiniz uygun isimde ve tipli alanlarla yazın
 - b. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın



3. Bir önceki soruda modellediğiniz köpeklerin sahipleri olması durumunu nasıl modellersiniz? Sadece alanları göz önüne alırsanız, sahiplerini nasıl bir sınıfla ve hangi alanlarla modellersiniz? Alanların isim ve tipleri ne olur?
 - a. Uygun sınıfı, seçtiğiniz alanları, uygun isim ve tiple yazın
 - b. Köpeklerle sahipleri arasındaki ilişkiyi 1-1 düşünerek modelleyin
 - c. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın



4. İki boyutlu bir ortamda bir noktayı nasıl modellersiniz? Belirlediğiniz alanlara varsayılan bir değer atamak konusunda ne düşünürsünüz?
 - a. Test sınıfının `main` metodunda, birisi merkezi (origin) temsil eden 3 nokta oluşturup aralarındaki uzaklıkları hesaplayın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra nesnelerin davranışlarını yani metotlarını inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

